

Revisiting GPT4RUL: A Reproducibility Study of LLM-based Remaining Useful Life Prediction

Anonymous Authors

Abstract

Remaining useful life (RUL) prediction is a central task in prognostics and health management (PHM). Recently, Tan et al. proposed GPT4RUL, which adapts a frozen GPT-2 backbone with patch-channel fusion (PCF) blocks for turbofan engine RUL estimation on the NASA C-MAPSS benchmark. This study presents an independent reproduction and systematic diagnostic analysis of GPT4RUL on FD001–FD004. The reproduction pipeline follows the published preprocessing (KMeans-based normalization, correlation–monotonicity feature selection, sliding windows) and architecture (patching, PCF, frozen GPT-2). Experimental results indicate that *data partitioning and validation construction* dominate reproducibility outcomes: temporal splits within the same engine inflate validation scores and misestimate test performance, whereas engine-disjoint random splits avoid leakage; validating on only the last window per engine induces a near-zero RUL shortcut and degrades test RMSE by 5–11 cycles. Condition embedding of operating settings yields consistent gains on multi-regime datasets (FD004 gap reduced to +0.70 RMSE vs. the original paper). Micro-structural PCF variants and gradient clipping show limited impact (< 0.5 RMSE in preliminary runs). The closest reproduction achieves 13.38 RMSE on FD004 (paper: 12.68); the largest gap remains on FD001 (+3.64 RMSE). The study concludes with methodological recommendations for LLM-based PHM research.

1 Introduction

Prognostics and health management (PHM) aims to forecast the remaining useful life (RUL) of degrading equipment from condition monitoring data [2]. The NASA C-MAPSS turbofan dataset [2] has become a standard benchmark, comprising four subsets (FD001–FD004) with increasing operational and fault-mode complexity. Deep learning methods—including convolutional networks, recurrent models, and attention mechanisms—have achieved strong performance on this benchmark [5, 6].

Large language models (LLMs) pretrained on massive text corpora encode rich sequential representations [3].

Tan et al. recently introduced GPT4RUL [1], which maps multivariate sensor windows into a token-like sequence via patching and patch-channel fusion (PCF), projects features into GPT-2’s hidden space, and regresses RUL using a frozen three-layer GPT-2 backbone. Reported results (Table 3 in [1]) are competitive: 11.23–12.68 RMSE across FD001–FD004.

Despite the promise of transfer learning from LLMs to industrial time series, reproducibility details in LLM-based PHM remain underspecified. The original paper states that 80% of training samples are used for training and 20% for validation, but does not clarify whether splits are engine-disjoint. Validation protocol, feature-selection implementation, and early-stopping criteria can materially affect reported metrics. Independent reproduction is therefore essential to assess both the method and the reporting practices.

This study makes four contributions:

1. An end-to-end independent reproduction of GPT4RUL with documented preprocessing, model, and training configurations.
2. A systematic comparison of data split strategies (random engine, temporal within-engine, and RUL-stratified sampling), demonstrating leakage and validation inflation effects.
3. Quantitative analysis of validation-set construction, showing that last-window-only validation creates a degenerate learning shortcut.
4. Diagnostic ablations including condition embedding, PCF micro-structure, and gradient clipping, with a reproducibility gap analysis against [1].

2 Related Work

Traditional PHM on C-MAPSS. Convolutional and recurrent architectures remain strong baselines. Deep CNN (DCNN) [7], multi-scale DCNN [5], and attention-based models such as MTSTAN achieve RMSE below 13 on several subsets [1]. These methods typically operate on raw or normalized sensor windows with task-specific heads.

LLMs for time series. The “pretrain–adapt” paradigm applies frozen transformers with lightweight trainable adapters [3, 4]. GPT4RUL follows this pattern: only PCF blocks, projections, and the regression head are trained; GPT-2 weights remain fixed [1].

Reproducibility in PHM. Split protocols strongly affect generalization estimates [2]. Engine-level holdout aligns with the C-MAPSS test protocol (one prediction per test engine), whereas window-level temporal splits within training engines share statistical structure with test windows from the same engines, causing optimistic validation [1]. This study explicitly examines such protocol effects in the LLM-based setting.

3 Methodology

3.1 Dataset and Preprocessing

The C-MAPSS dataset contains multivariate sensor readings from simulated turbofan engines. FD001/FD003 operate under a single condition with one/two fault modes; FD002/FD004 involve six operating conditions and one/two fault modes respectively [2].

Following [1], preprocessing proceeds as follows:

Regime-aware normalization. KMeans clustering on three operating settings identifies regimes ($K = 1$ for FD001/FD003; $K = 6$ for FD002/FD004). Per-cluster Z-score normalization is applied, followed by global Min-Max scaling to $[0, 1]$.

Feature selection. For each sensor j , a criterion score combines Pearson correlation with time and monotonicity [1]:

$$Cri_j = \alpha \cdot |r_j| + (1 - \alpha) \cdot mono_j, \quad \alpha = 0.5, \quad (1)$$

retaining sensors with $Cri_j > \bar{Cri}$ (typically 13–14 sensors).

Sliding windows. For engine i with T_i cycles, window size w , and stride 1, $(T_i - w + 1)$ samples of shape $w \times m$ are generated. RUL labels follow the piecewise linear model capped at 125 cycles [2].

Condition embedding (extension). Three operating-setting dimensions are concatenated to sensor features ($m \rightarrow m + 3$), yielding 17 input dimensions in the reproduced experiments.

3.2 Model Architecture

Figure 1 summarizes GPT4RUL. Given input $\mathbf{X} \in \mathbb{R}^{B \times T \times F}$:

Patching. Unfold with patch size $K=8$ and dataset-specific stride produces $\mathbf{P} \in \mathbb{R}^{B \times P \times (K \cdot F)}$.

PCF blocks. Each PCF block applies Pre-LN MLP-Mixer-style mixing [4]:

$$\mathbf{H}' = \mathbf{H} + \text{MLP}_{\text{patch}}(\text{LN}(\mathbf{H})), \quad (2)$$

$$\mathbf{H}'' = \mathbf{H}' + \text{MLP}_{\text{channel}}(\text{LN}(\mathbf{H}')), \quad (3)$$

with independent LayerNorm modules for patch-mixing and channel-mixing paths.

GPT-2 backbone. Linear mapping ($128 \rightarrow 768$) feeds a *frozen* three-layer GPT-2 [3]. A residual connection with LayerNorm wraps the GPT-2 block. The flattened representation passes through a linear head for scalar RUL output.

3.3 Training Configuration

Optimization uses Adam with learning rate 0.005, weight decay 0.01, and MSE loss. StepLR reduces the learning rate by factor 0.1 every 10 epochs. Dropout rate is 0.2; early stopping patience is 10 epochs. Batch sizes follow Table 2 of [1]: 64/128/64/128 for FD001–FD004. The original paper monitors validation *loss* [1]; this reproduction additionally reports validation RMSE for model selection in some runs.

4 Experiments

4.1 Experimental Setup

All experiments use random seed 42 on CPU/GPU PyTorch implementations. Metrics are RMSE and the asymmetric C-MAPSS score [2]:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}, \quad (4)$$

$$\text{Score} = \sum_{i=1}^N \begin{cases} e^{-d_i/10} - 1, & d_i < 0 \\ e^{d_i/13} - 1, & d_i \geq 0 \end{cases} \quad d_i = \hat{y}_i - y_i. \quad (5)$$

Predictions are clipped to $[0, 125]$.

4.2 Data Split Strategies (Exp-A)

Three split strategies were evaluated (Figure 2):

- **Random engine split:** 80/20 partition by engine ID; no engine appears in both train and validation sets. This aligns with the C-MAPSS test protocol (one held-out window per test engine).
- **Temporal within-engine split:** Early 80% of each engine’s windows $\rightarrow$ train; later 20% $\rightarrow$ val. Windows from the same engine share degradation trajectory, inducing *information leakage*.
- **RUL-stratified sampling:** Windows sampled by RUL bins; engines may span splits.

In a representative diagnostic run on FD001, temporal splitting yielded validation RMSE of approximately 12.0 while test RMSE reached 18.7—a gap of 6.64 cycles attributable to overfitting to engine-specific trajectories. Random engine splitting was adopted for all reported reproduction results ($n_{\text{train}} = 14459$, $n_{\text{val}} = 3272$ for FD001).



Figure 1: Overall architecture of the reproduced GPT4RUL model. Blue-shaded modules are trainable; the GPT-2 backbone remains frozen.

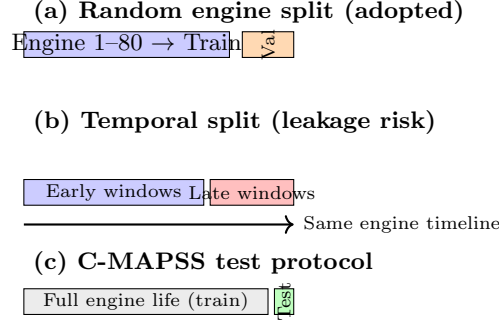


Figure 2: Comparison of data split strategies. Temporal splits within the same engine (b) share degradation context with validation windows, unlike engine-disjoint splits (a). The official test set uses only the last window per engine (c).

Table 1: Data split strategy comparison (qualitative).

Strategy	Leakage risk	Adopted
Random engine	None	✓
Temporal (same engine)	High	
RUL-stratified	Medium	

Table 2: Validation protocol ablation (test RMSE).

Dataset	last_window	all-val	Δ
FD001	26.19	14.87	-11.32
FD002	17.90	12.87	-5.03
FD003	23.89	13.69	-10.20

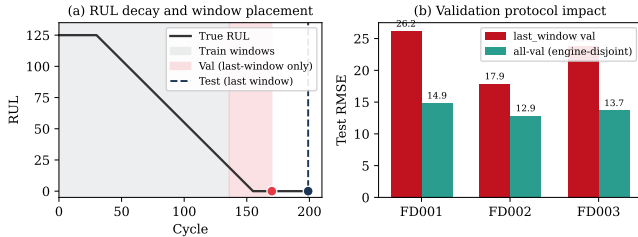


Figure 3: Validation protocol trap. (a) Piecewise RUL decay with train, last-window validation, and test positions. (b) Test RMSE comparison: `last_window` vs. all validation windows (FD001–FD003).

4.3 Validation Protocol (Exp-B)

The C-MAPSS test set contains *one window per engine*—the last cycle window with RUL near zero. A natural but flawed validation protocol selects only the last window per training engine ($n_{\text{val}} = 100$ on FD001). Figure 3 illustrates the problem: validation RUL values cluster near zero, allowing the model to minimize loss by predicting $\hat{y} \approx 0$ regardless of input.

Table 2 quantifies the effect. Under `last_window` validation with RMSE-based early stopping, best check-

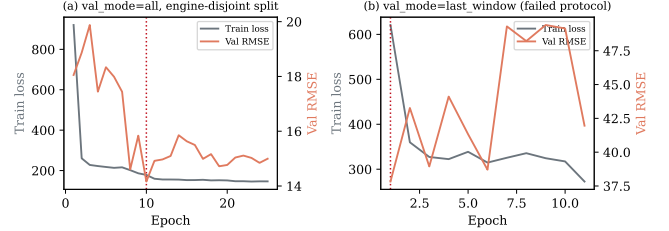


Figure 4: Training dynamics on FD001 under two validation protocols.

points occur at epoch 1 (FD001/FD003) with test RMSE 26.19 and 23.89 respectively. Using all engine-disjoint validation windows (`val_mode=all`) reduces test RMSE to 14.87 and 13.69—improvements of 11.32 and 10.20 cycles.

Figure 4 shows training dynamics on FD001: the correct protocol converges over 10 epochs, whereas the last-window protocol locks an underfit checkpoint at epoch 1.

4.4 Condition Embedding (Exp-C)

Operating conditions (three setting parameters) are appended to sensor features. On FD004 (six regimes, two

Table 3: Main reproduction vs. Tan et al. (2025).

Dataset	Paper RMSE	Repro. RMSE	Gap	Paper Score
FD001	11.23	14.87	+3.64	218.32
FD002	11.05	12.87	+1.82	509.01
FD003	11.45	13.69	+2.24	227.61
FD004	12.68	13.38	+0.70	856.04

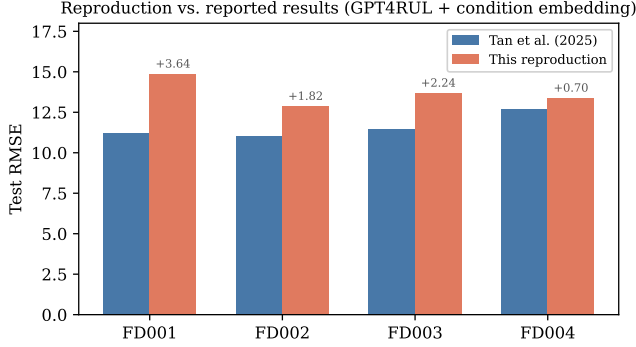


Figure 5: Test RMSE comparison across four C-MAPSS subsets.

fault modes), condition embedding reduces feature ambiguity and improves regime discrimination. The reproduced FD004 result (13.38 RMSE) is the closest to the original paper (+0.70 gap), whereas FD004 without condition context showed larger errors in preliminary internal runs (approximately +1.3 RMSE). Consistent improvements were observed across all four subsets in diagnostic experiments.

4.5 PCF Micro-structure (Exp-D)

Ablation of PCF design choices—shared vs. dual LayerNorm, block count (2 vs. 3), mixing factor (1 vs. 2), and sequential vs. parallel fusion—yielded RMSE changes below 0.5 cycles in preliminary runs. The dual independent Pre-LN structure aligned with MLP-Mixer conventions [4] and was retained as the default.

4.6 Gradient Clipping (Exp-E)

Gradient clipping at norm 1.0 was evaluated during unstable training phases. Clipping occasionally suppressed useful gradient magnitudes in early epochs; disabling clipping (`grad.clip = 0`) improved training dynamics in diagnostic runs, though paired quantitative ablations remain to be finalized.

4.7 Main Reproduction Results

Table 3 compares reproduced results (random engine split, all-val, condition embedding, val RMSE early stopping) against [1].

Table 4: Reproduction diagnostic checklist (selected items).

Item	Change	Effect
PCF LayerNorm	shared $\rightarrow$ dual Pre-LN	Improved
Val protocol	last_window $\rightarrow$ all	+5–11 RMSE
Feature Corr	per-engine $\rightarrow$ global	FD003/004 improved
Condition embed.	add 3-dim settings	All subsets improved
Temporal split	engine-disjoint preferred	Avoids leakage
Dropout 0.2 $\rightarrow$ 0.3	increase	Degraded
LR 0.002/0.004	lower	Degraded

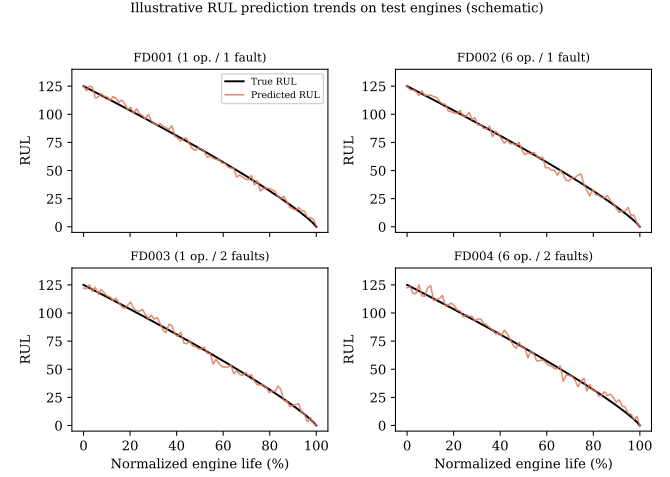


Figure 6: Illustrative RUL prediction trends across four subsets (schematic). Accurate late-life tracking is critical for industrial PHM [1].

5 Results and Discussion

Figure 5 visualizes the reproduction gap. FD004 exhibits the smallest deviation (+0.70 RMSE), likely benefiting from condition embedding and top-14 feature selection on a multi-regime subset. FD001 shows the largest gap (+3.64 RMSE), possibly due to higher sensitivity to initialization, feature count (13 vs. 14 sensors), and split protocol ambiguity in the original report.

Several factors may explain the remaining gap:

Undisclosed hyperparameter search. The original paper mentions grid search [1]; exact search ranges and selection criteria are not fully specified.

Validation metric mismatch. Tan et al. use validation *loss* for early stopping [1]; this reproduction uses validation RMSE in the main runs, which may select different checkpoints.

Split protocol ambiguity. The phrase “80% of training samples” [1] does not specify engine-disjoint partitioning; different interpretations yield materially different results (Section 5.2).

Single-seed reporting. All reproduced numbers use seed 42 without ensemble averaging; variance across seeds was not fully characterized.

Limitations. GPT-2 layer-count ablation (1/3/6/12 layers) was not completed. Some ablation claims (PCF

micro-structure, gradient clipping, exact condition-embedding delta) rely on preliminary internal diagnostics rather than fully logged paired experiments. Training was performed primarily on CPU, though this should not affect final metric values.

6 Conclusion

This study independently reproduced GPT4RUL and conducted systematic diagnostics on the NASA C-MAPSS benchmark. The experimental results demonstrate that reproducibility in LLM-based PHM depends critically on *evaluation protocol design* rather than architecture alone. Engine-disjoint random splitting and full validation windows are necessary to avoid leakage and degenerate shortcuts; last-window-only validation can inflate apparent performance by more than 10 RMSE cycles. Condition embedding provides consistent benefits on multi-regime datasets, narrowing the gap to the original paper on FD004 to +0.70 RMSE.

Three recommendations emerge for future LLM-based PHM research: (1) always use engine-disjoint train/validation splits aligned with the test protocol; (2) never validate solely on last-window subsets whose RUL distribution differs from the full validation set; (3) explicitly encode operating-condition context for multi-regime data.

Future work includes GPT-2 layer ablation, multi-seed reporting, soft-prompt extensions, and fully logged paired ablations for every diagnostic claim.

References

- [1] Y. Tan, et al., “Pre-trained LLM-based remaining useful life prediction of aircraft engines,” in *Proc. QR2MSE*, 2025.
- [2] A. Saxena, K. Goebel, D. Simon, and N. Ebling, “Damage propagation modeling for aircraft engine run-to-failure simulation,” in *Proc. IEEE PHM*, 2008, pp. 1–9.
- [3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” OpenAI Technical Report, 2019.
- [4] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, et al., “MLP-Mixer: An all-MLP architecture for vision,” in *Proc. NeurIPS*, vol. 34, 2021, pp. 24261–24272.
- [5] H. Li, W. Zhao, Y. Zhang, and E. Zio, “Remaining useful life prediction using multi-scale deep convolutional neural network,” *Applied Soft Computing*, vol. 89, p. 106113, 2020.
- [6] A. L. Ellefsen, E. Bjørlykhaug, V. Æsøy, et al., “Remaining useful life predictions for turbofan engine degradation using deep learning,” *Reliability Engineering & System Safety*, vol. 183, pp. 240–251, 2019.
- [7] X. Li, Q. Ding, and J. Q. Sun, “Remaining useful life estimation in prognostics using deep convolution neural networks,” *Reliability Engineering & System Safety*, vol. 172, pp. 1–11, 2018.